



Operating Systems

[1500WETOPS]

José Oramas

Very Important PSA

OS Exam: January 29th (GBC T105)

- Theory Exam
 - Close book, no oral component
 - Shorter than example in BB
- Exercises Exam
 - Open book (course content)
 - Prinouts of course notes, slides and exercises solved.



Filesystems

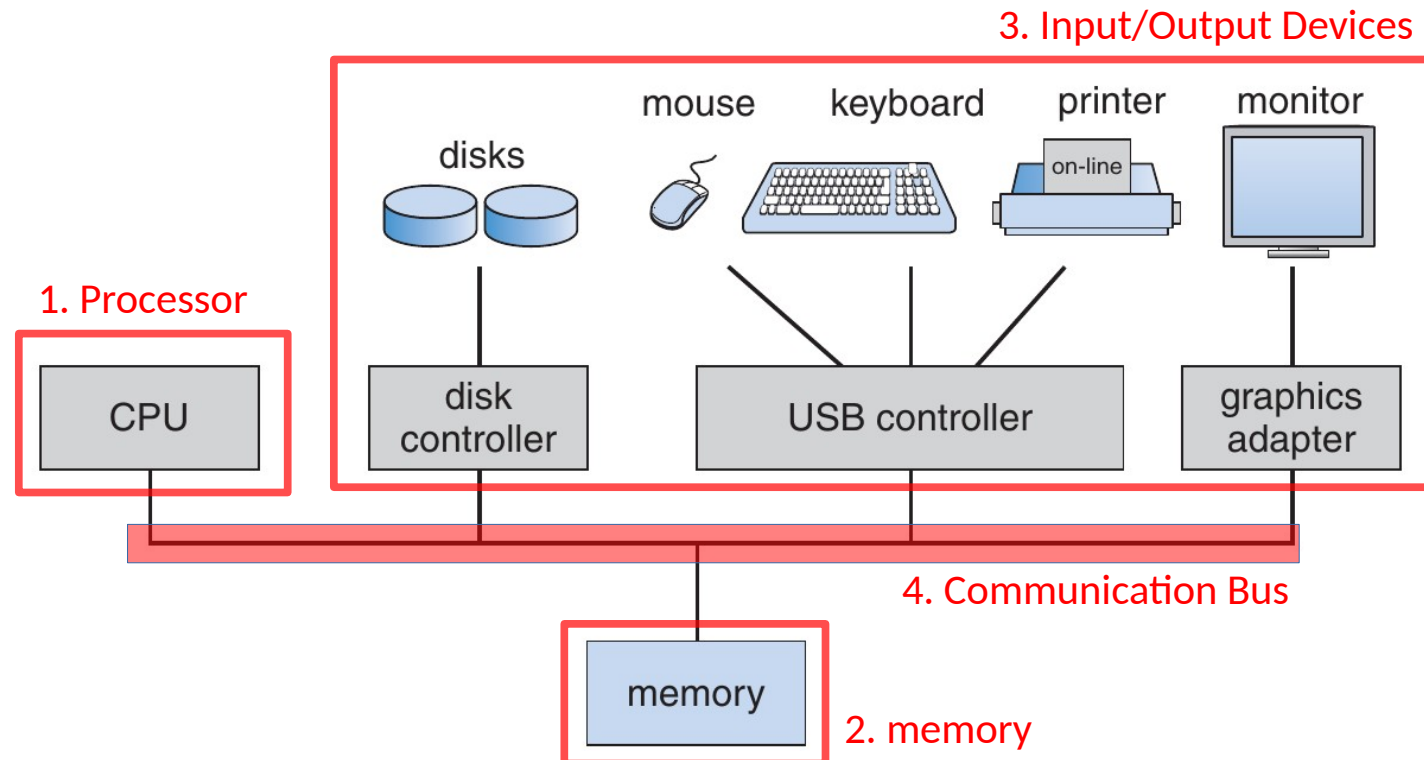
[Organizing Content in Storage Devices]

José Oramas

Operating systems

A Modern Computer System

- Main components



Filesystems

Introduction

- Common attributes
 - File name (max. length)
 - Type
 - Location (I/O device + device location)
 - Size
 - Protection (read, write and output permission)
 - Time, date and user id.
- This information is brought together in the directory structure

Filesystems

File Operations

- Creating a file
 - free-space management
- Read or write file
 - File location (initially via file name)
 - Using current-file pointer (location is not a parameter)
- File Search
 - Customize current-file pointer
- Delete file
 - file location, free-space management

Filesystems

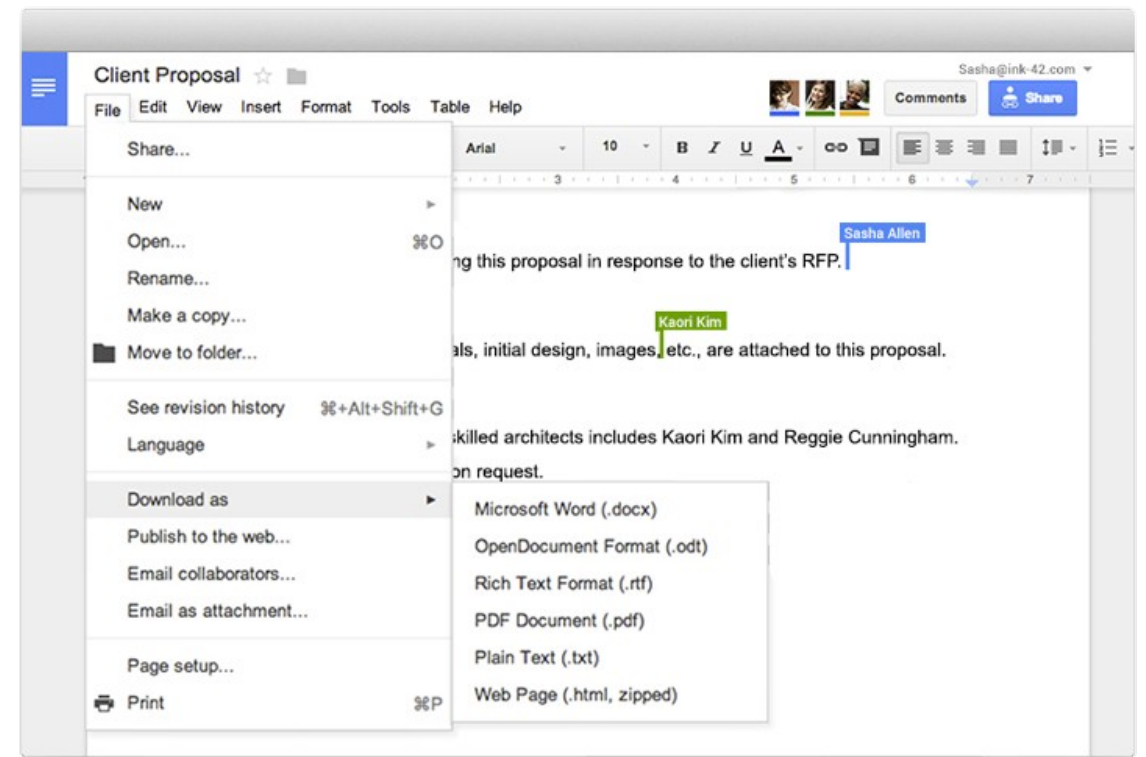
Open-File Table

- Purpose: avoid multiple searches of file directory structure
- Contents: contains directory entries of the opened files
- Index to table is returned as a pointer in an open operation
- Handle pointer for all future I/O operations on the file

Filesystems

Open-File Table

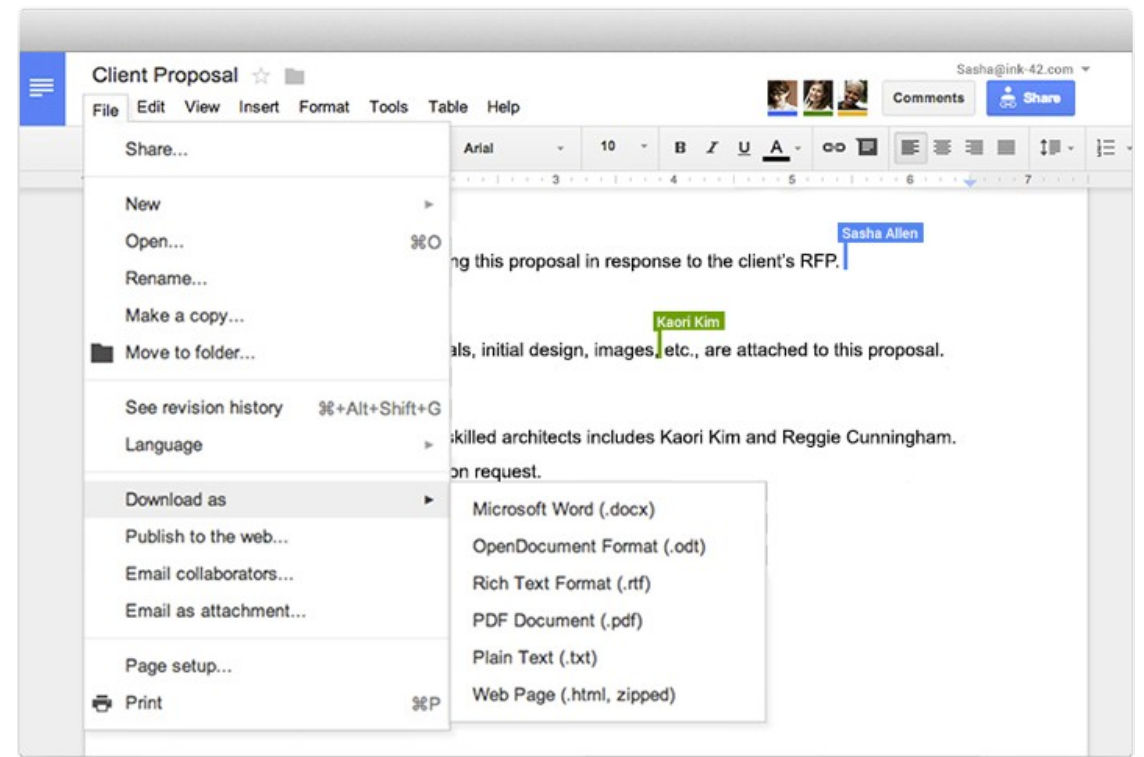
- Multiprogramming environment
- **Per-process** open-file table
 - Contains entry per opened file
 - Every entry has a current-file pointer, as well as a pointer to the system-wide table



Filesystems

Open-File Table

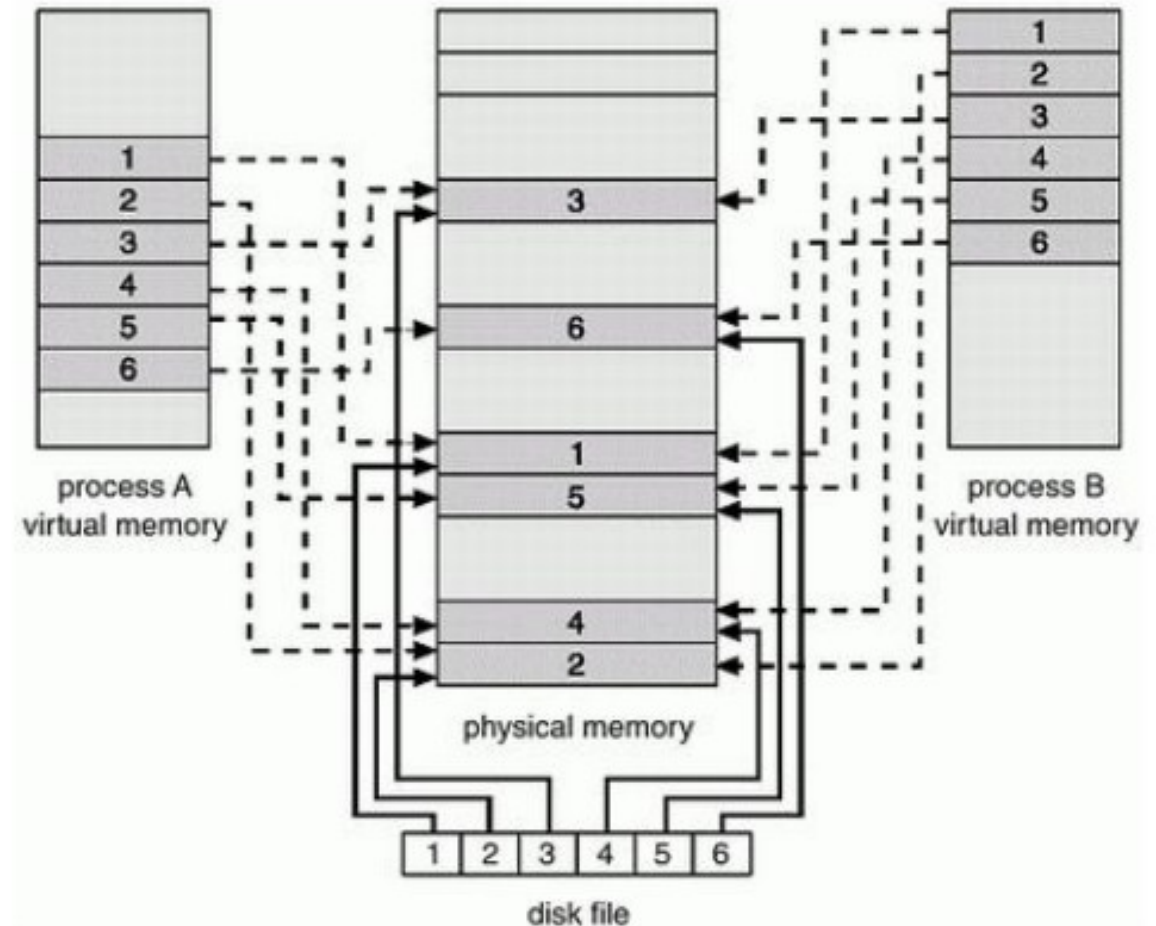
- Multiprogramming environment
- **Per-process** open-file table
 - Contains entry per opened file
 - Every entry has a current-file pointer, as well as a pointer to the system-wide table
- **System-wide** open-file table
 - Contains file info which is identical for all processes (file location, size, data, protection info)
 - Open file counter counts the number of refs to an entry (open +1, close -1, ==0 delete)



Filesystems

Memory-Mapped Files

- Locking part of a file in virtual memory
- Fast access
- At close operation write back to disk
- Easy file sharing via virtual page tables



Filesystems

File Types

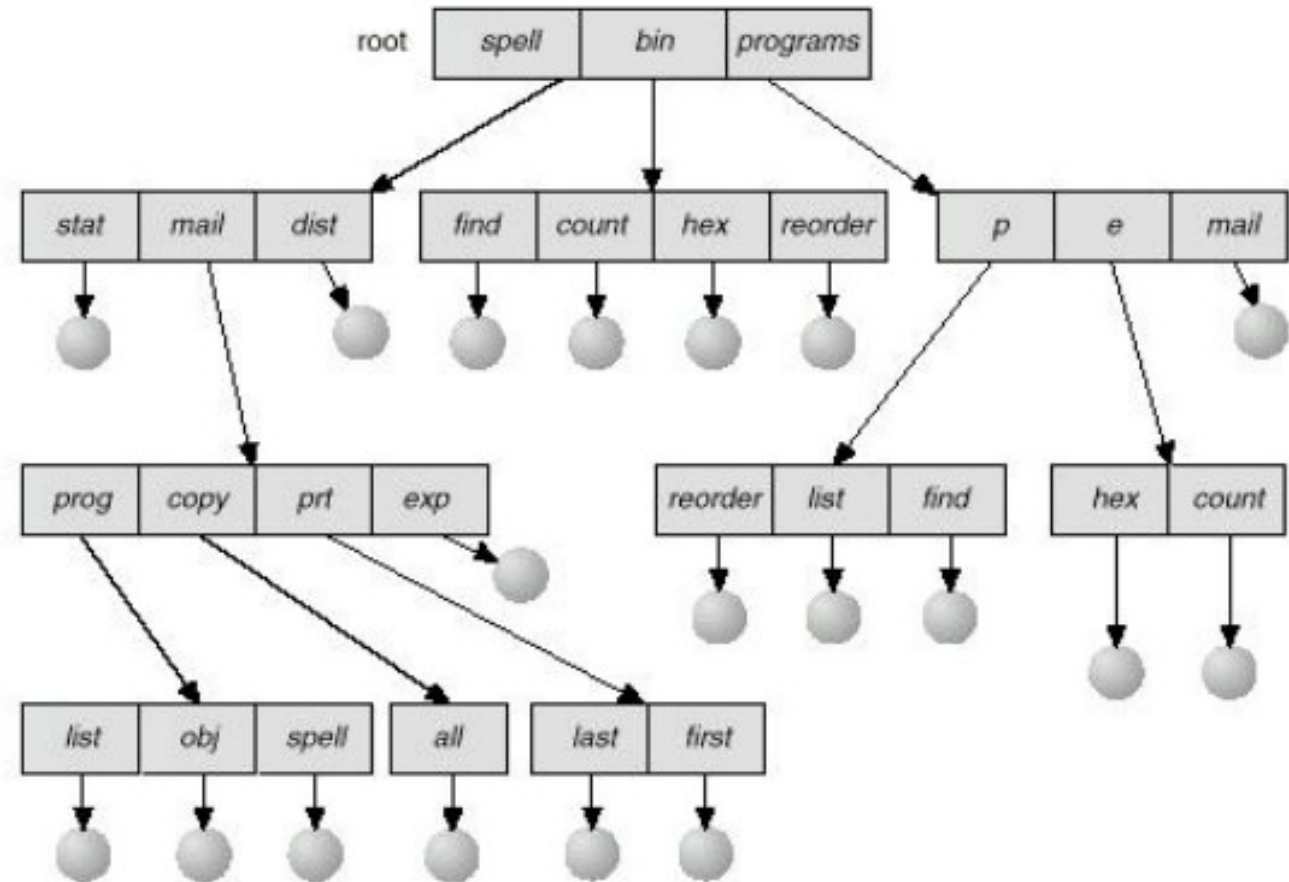
- Most OS support
 - Very small collection of file types (at least executable)
 - Recognizable via extension
- Often trade-off between complexity and ease of use, e.g.
 - Print binary files
 - Customizing GUI labels (Mac)
 - Unnecessary compilation of object files

Directory Structure

Filesystems

Directory Structure per Partition

- Directory file per node
- Entry per subdirectory and per file
- Path in tree gives unique file name
- Browse tree when opening file



Filesystems

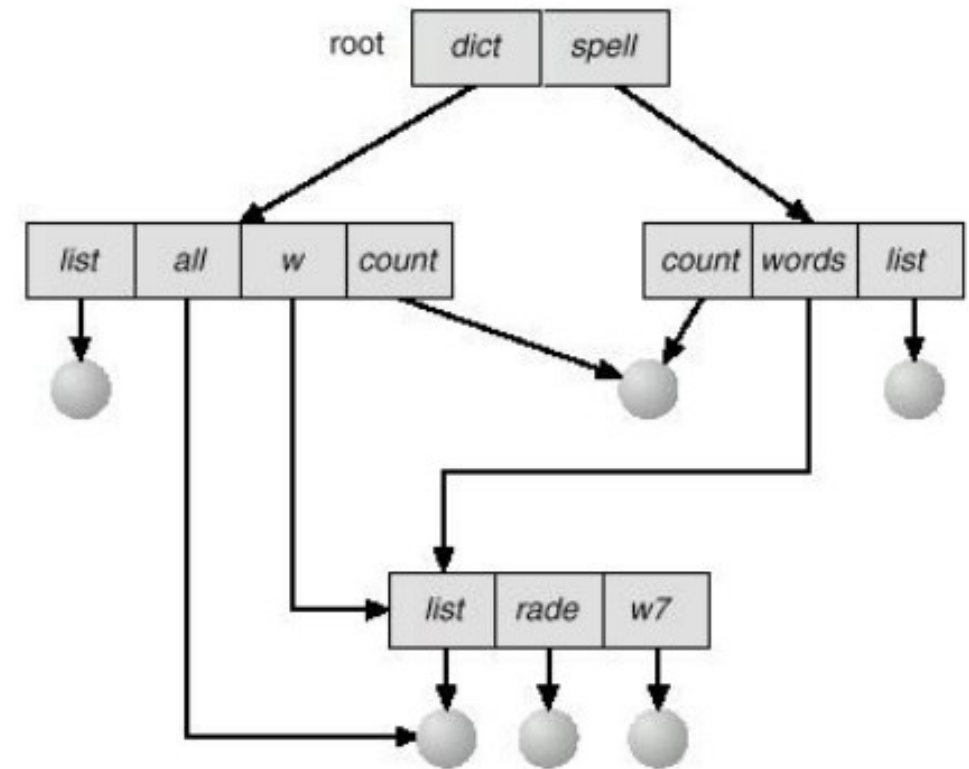
Shell & Search Path

- Working from within the shell with short file names & relative path names
 - In current directory
 - In directory which is part of search path
- MS-DOS: first search local directory file, then search path
- UNIX: searches only search path, of which current directory is part of by default
- Search path useful for simple file reorganizations

Filesystems

Directory Structure

- Fast access to files located elsewhere without using a search path
- **Symbolic links** (directory entry contains path name to actual file)
- No unique path names
- Potentially closed paths, dangerous for routines running down structure
 - Ignore symbolic links



Filesystems

Graph Structure

- Choices:
 - If we follow symbolic link and then go to parent directory, where do we end up? (shell dependent)
 - Can we just delete a file if there are still references to it
 - Can keep counter with number of refs and only delete when counter gets zero
 - **Hard links** in UNIX keep counter, **soft links** do not guarantee the existence of the file

Filesystems

Directory Structure – Implementation

- Linear list:
- Binary tree (Ordered list):
- Hash function

Filesystems

Directory Structure – Implementation

- Linear list:
 - Always search the whole list (even when creating to check uniqueness)
 - Delete we can mark entry as open or put the last entry in its place
- Binary tree (Ordered list):
- Hash function

Filesystems

Directory Structure – Implementation

- Linear list:
 - Always search the whole list (even when creating to check uniqueness)
 - Delete we can mark entry as open or put the last entry in its place
- Binary tree (Ordered list):
 - Binary search method gives logarithmic search time
 - More work when creating and deleting files
- Hash function

Filesystems

Directory Structure – Implementation

- Linear list:
 - Always search the whole list (even when creating to check uniqueness)
 - Delete we can mark entry as open or put the last entry in its place
- Binary tree (Ordered list):
 - Binary search method gives logarithmic search time
 - More work when creating and deleting files
- Hash function (how many results?)
 - Result points to linear list

Access & Allocation Methods

Filesystems

Access and Allocation Methods

- Access methods
 - **Sequential access:** read records from first to last, add info at the end (editors, compilers, images, etc.)
 - **Direct or relative access:** give the number of the logical record as a parameter
 - Search method needed that converts input data to logical record number
- Allocation Methods: **Continuous, Linked & Indexed**

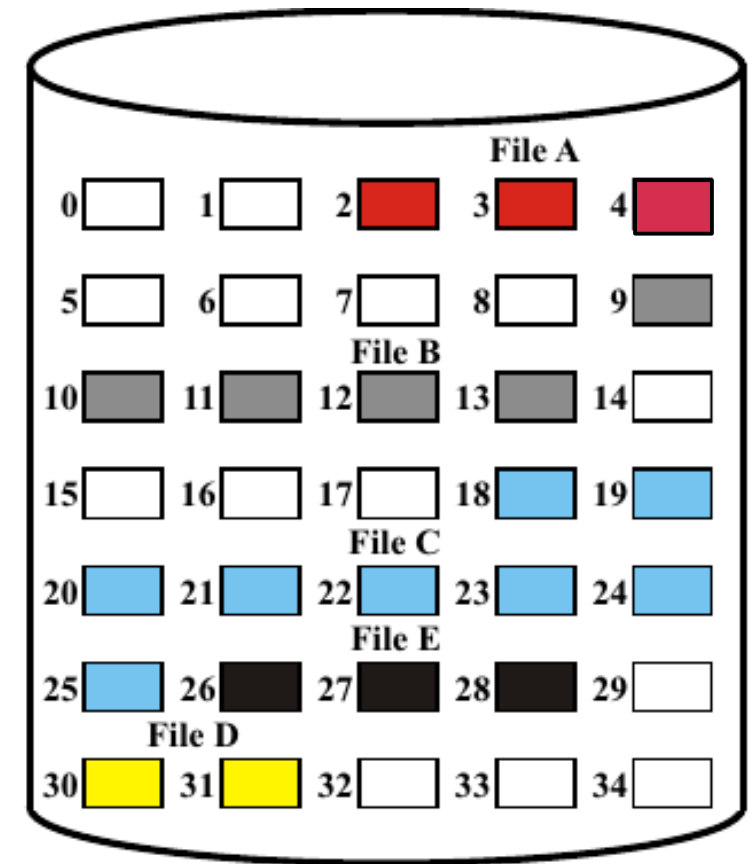
Filesystems

Continuous Allocation

- Each file has number of consecutive blocks
- *Info = (starting block , length)*
- Fast sequential and direct access (very low seek times)
- Placement: First-fit, Best-fit, Next-fit, Worst- fit
- Quite analogous to dynamic partitioning

Directory

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3



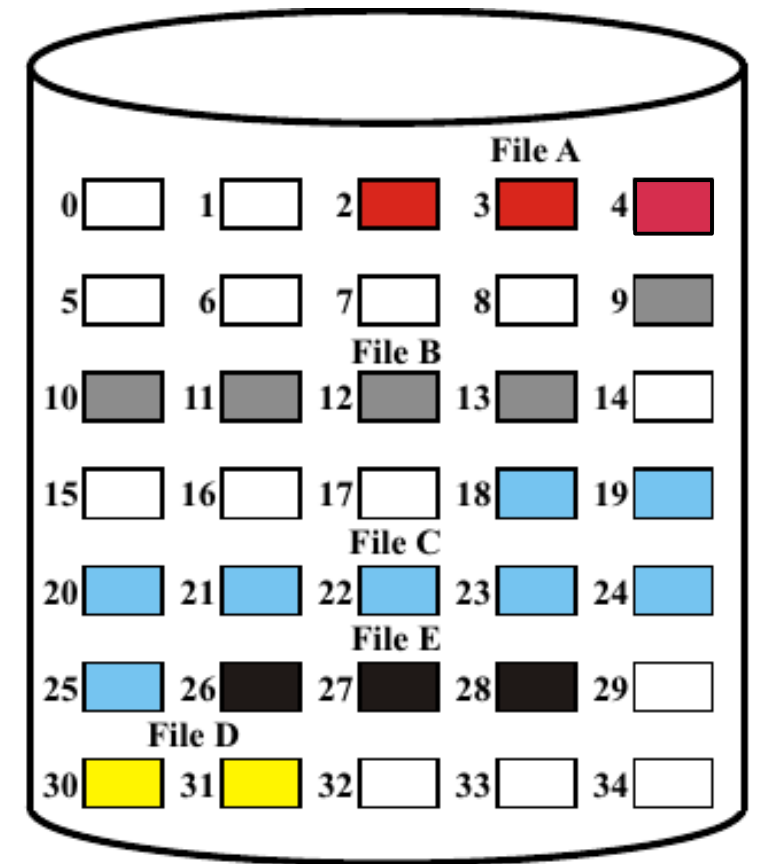
Filesystems

Continuous Allocation

- **Problem:** Dynamically increasing file size
 - or upper limit (internal fragmentation)
 - or size of extension
 - (additional set of blocks elsewhere)
 - or move data frequently
- Defragmentation routine often available

Directory

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3



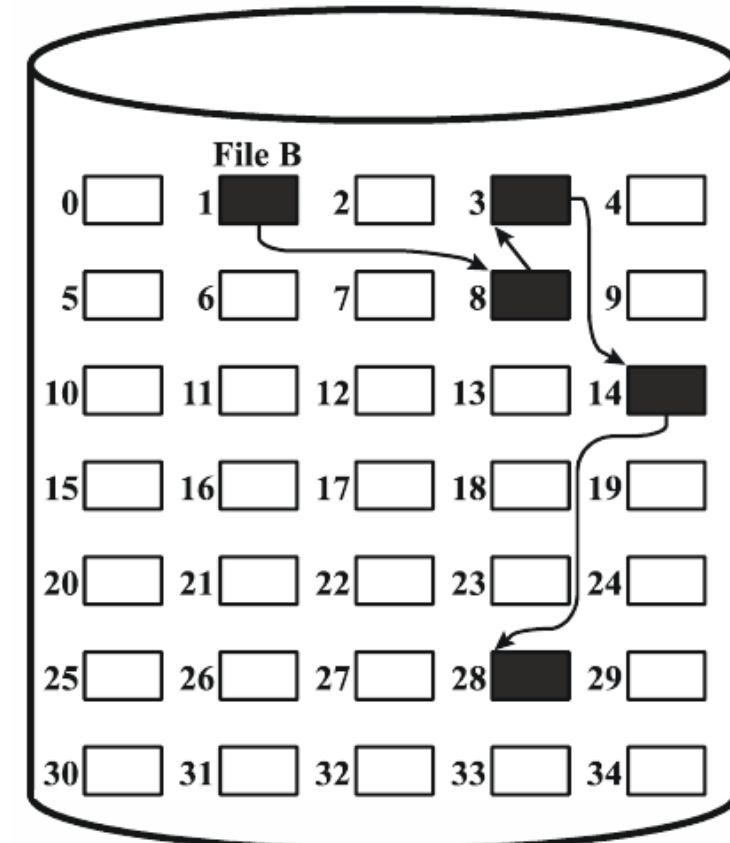
Filesystems

Linked (or Chained) Allocation

- Each file forms a linked list of random blocks
- *Info = (starting block , length)*
- Fairly fast sequential accesses (considerable seek times)
- Slow direct access
- Positioning: for each block we call free-space algorithm

Directory

File Name	Start Block	Length
...	...	...
File B	1	5
...	...	...

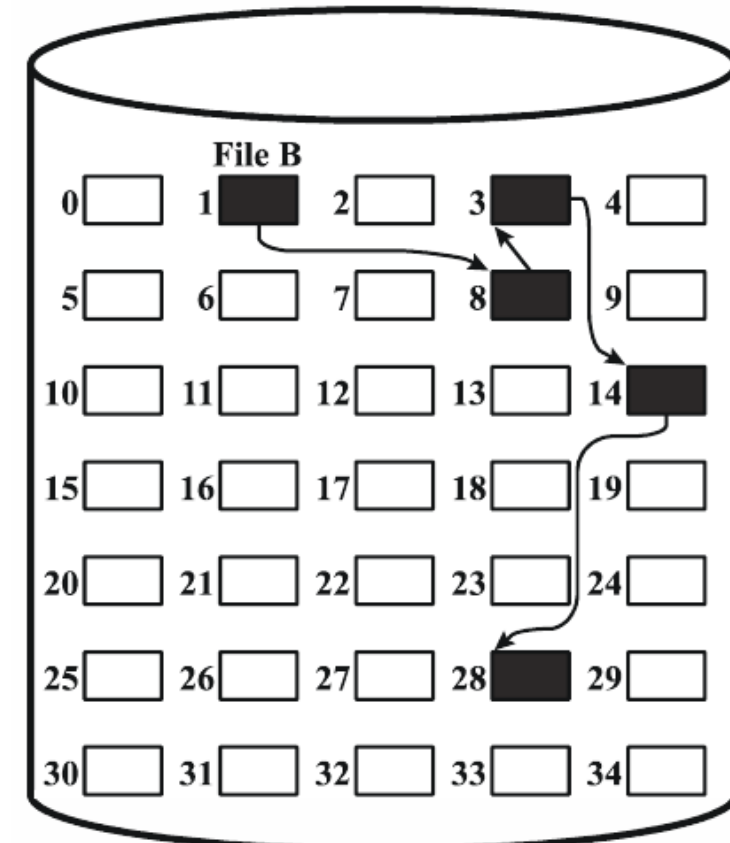


Filesystems

Linked Allocation

- Disadvantages:
 - Possibly slow access
 - Lose number of bytes per block for pointer
- Partially resolve by forming clusters:
 - Each cluster consists of a fixed number of blocks
(e.g. 4 blocks)
 - Gives faster access
 - Less loss of capacity due to pointer

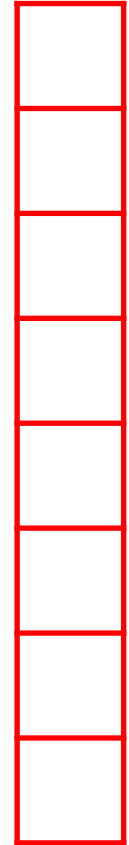
Directory		
File Name	Start Block	Length
...	...	...
File B	1	5
...	...	...



Filesystems

File Allocation Table (FAT)

- Variant of linked allocation
- FAT contains entry per disk block
 - An entry contains number of next block (belonging to the same file)
 - 0-value indicates that block is unused
- Starting block number is indexed to FAT
- Free-space algorithm selects first block with 0 value



Filesystems

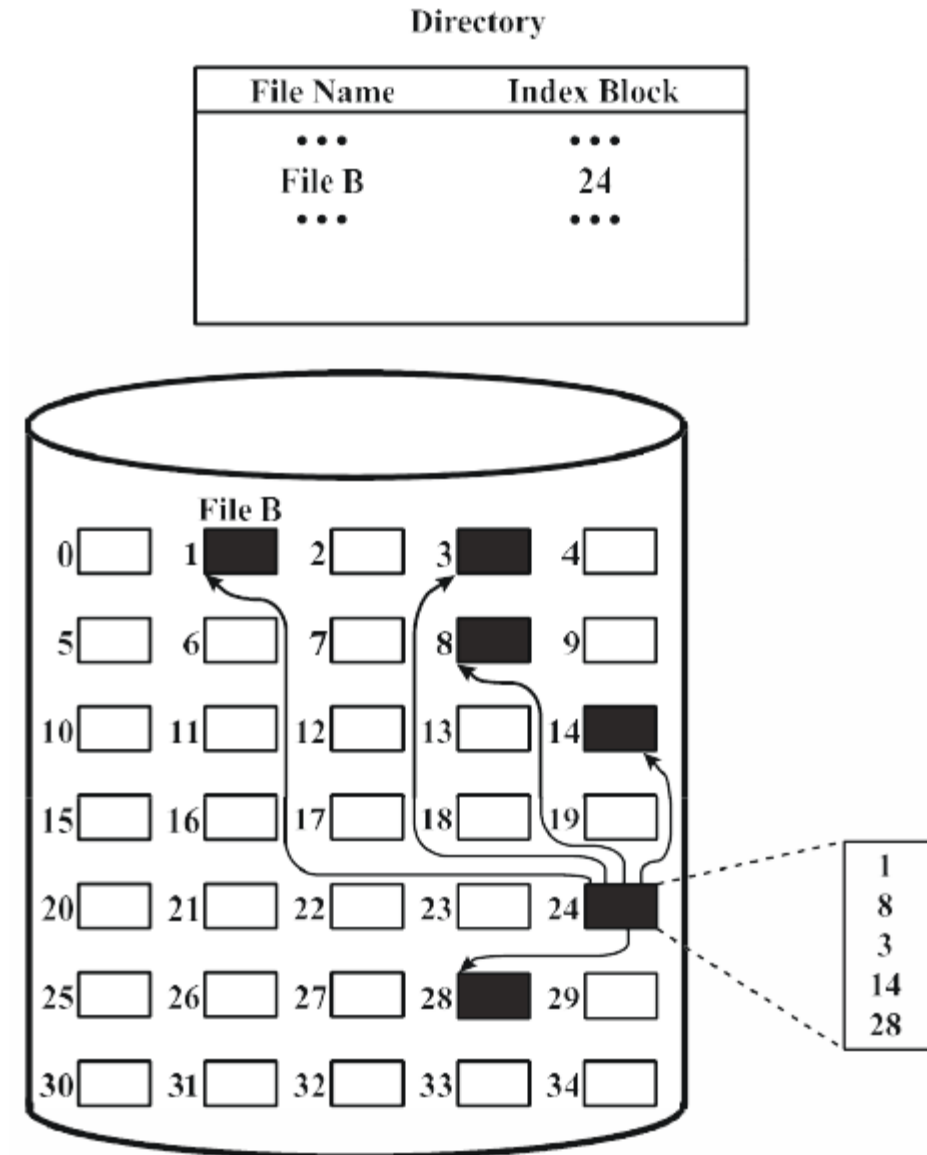
File Allocation Table (FAT)

- Advantages:
 - Direct access: FAT is on the start of the disk, making direct access much faster than linked access (low seek time)
 - Sequential access: often FAT partly in cache, so with sequential access we can avoid moving back and forth between FAT and data
- FAT used with MS-DOS, OS/2, USB flash drives

Filesystems

Indexed Allocation

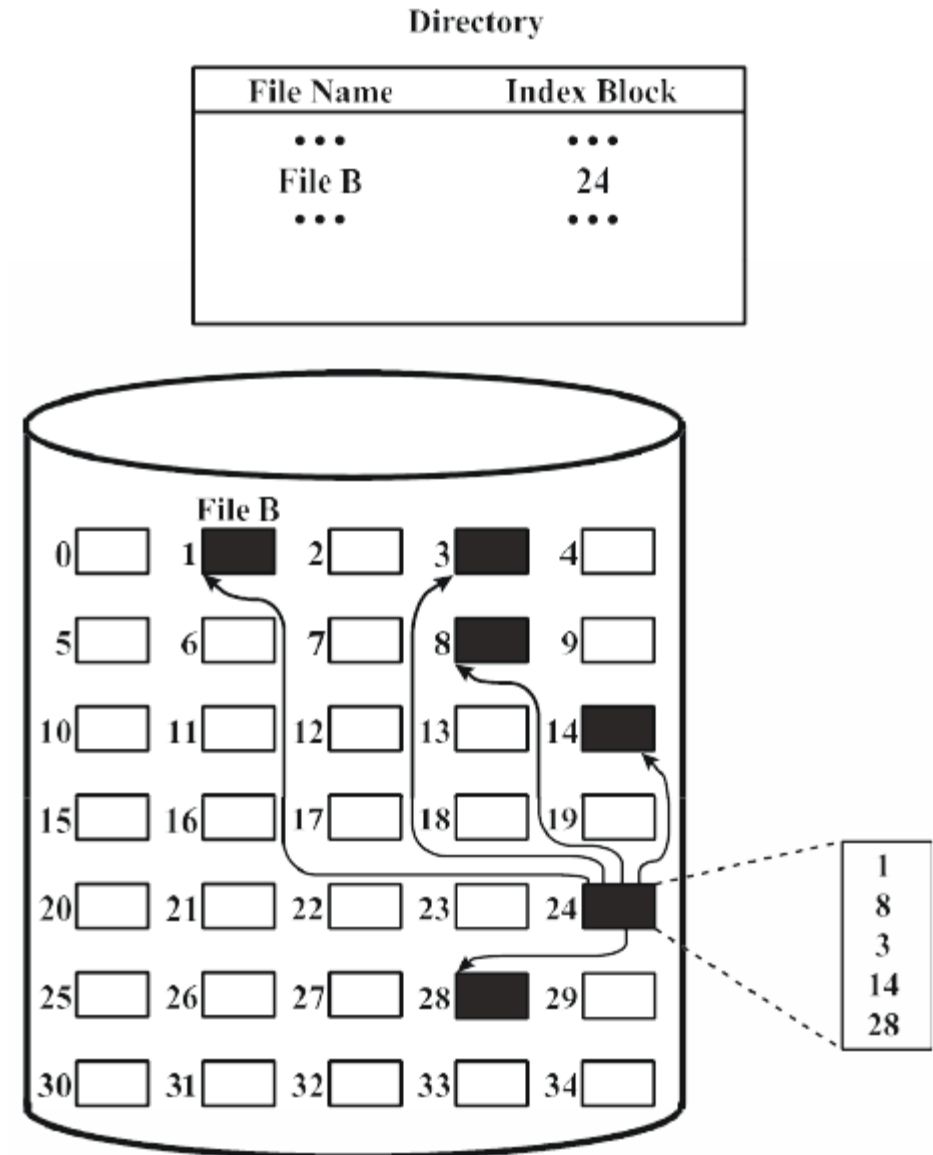
- Each file has number of random blocks + index block
- *Info = index block*
- Less fast sequential access (seek times)
- Very fast direct access (2 pointers)
- Positioning: for each block we call free- space algorithm
- Quite analogous to paging



Filesystems

Indexed Allocation

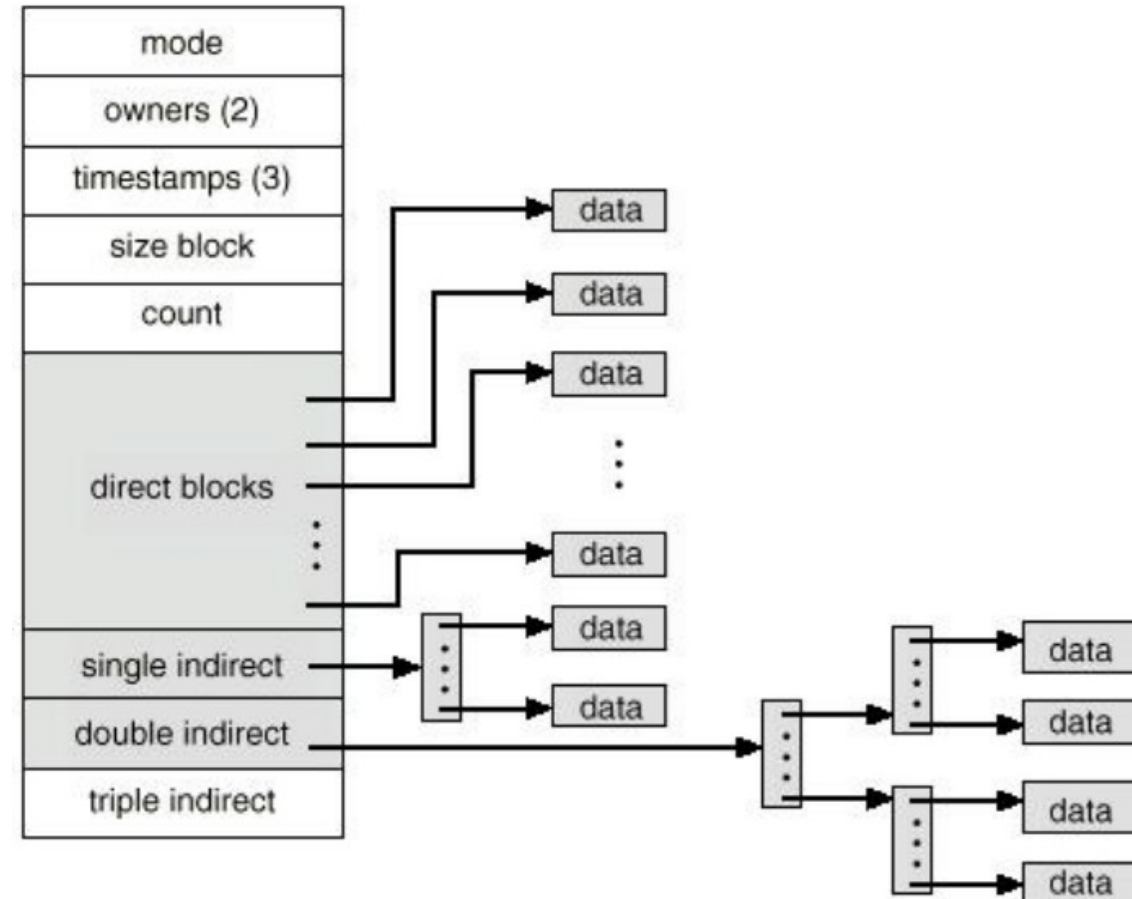
- How big should index be?
 - At least 1 block (disadvantage for small files)
 - To support large files multiple blocks required
 - As the linked list becomes long direct access gets slow for blocks at the end.
 - Multilevel index: completely analogous to *multilevel page tables* when paging memory



Filesystems

UNIX inodes (index-node)

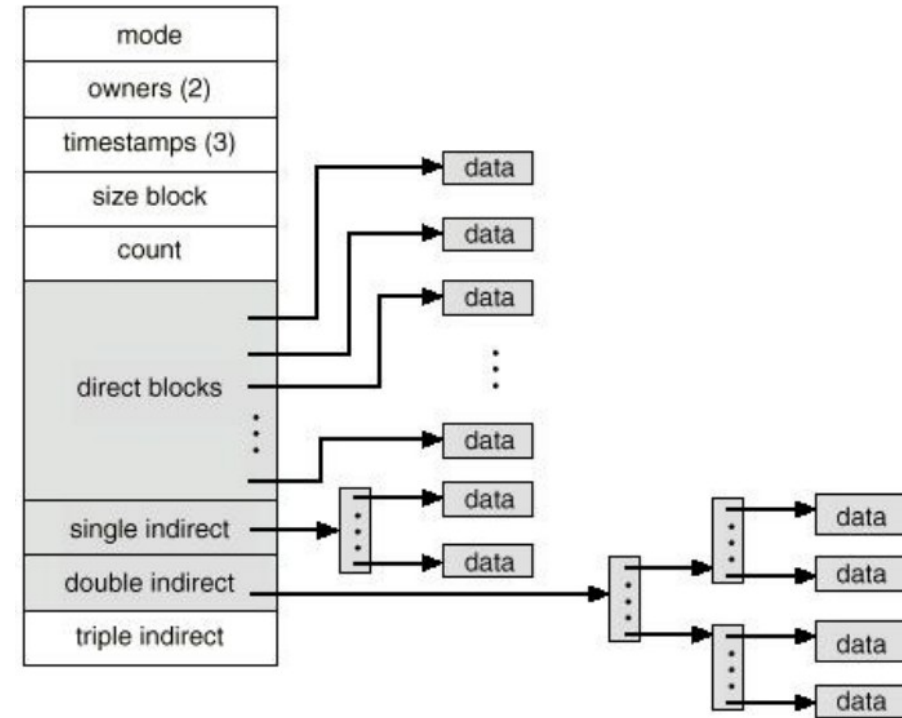
- inode per file/directory
- Directory only translates file name into inode number
- Mode: protection
- Count: hard link support
- Form of index allocation:
 - Directly for blocks 1 to 12
 - Then indirectly: first single, then double



Filesystems

UNIX inodes

- Example: 1K blocks, 4 byte pointers
 - 10 direct: files up to 10K direct access (1 pointer)
 - 256K via single indirect (2 pointers)
 - 64M via double indirect (3 pointers)
 - 16G via triple indirect (4 pointers)
- Placement of inodes on disk is important
(best as close to data as possible)



Free-Space Management

Filesystems

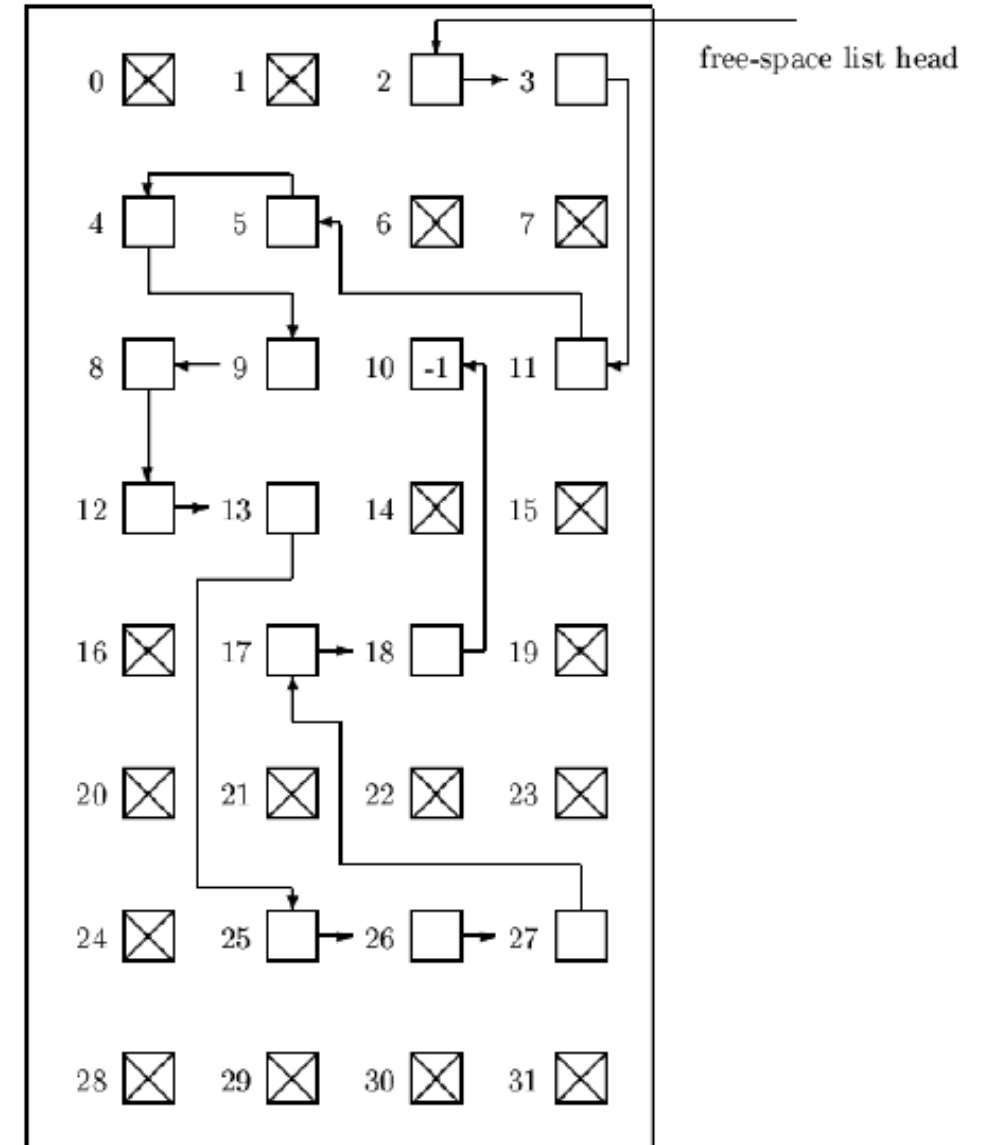
Free-space Management

- Bit vector (bitmap) approach:
 - Bit tracking per block, a 1 indicates that the block is free
 - Vector is read word by word in the processor and if word $\neq 0$ then the position of the first 1 is determined (via hardware optimizations)
 - Analog also quickly find sequence of bits = 1
 - Disadvantage: bit vector becomes too long in today's systems

Filesystems

Free-space Management

- Linked List Approach
- Very fast allocation
- In file deletion:
 - Add to the back of the list
 - Fast at linked allocation
- Difficult to support continuous allocation
- At FAT integral part of table



Filesystems

Free-space Management

- Indexing solution
 - Index to free blocks
 - Again, possibility of multilevel system
 - Attempt to offer a single index per series of consecutive free blocks

Journaling

Filesystems

Journaling

- **Goal:** Keep the file system consistent after a crash or power outage.
- Use case: Second Extended (ext2) File System from Linux.
 - Disk blocks are divided into groups (inodes + data blocks)
 - Each group contains two bitmaps indicating the usage of inodes and datablocks.

Filesystems

Data Inconsistency VS. Nonsensical Data

- Consider a 1-block data file

(the 1st direct pointer in the inode points to the data block)

- Adding a datablock to this file involves three changes:
 - 1) Data must be written to data block
 - 2) Direct pointer from inode should point to the data block.
 - 3) Update the data block usage bitmap.

Filesystems

Data Inconsistency VS. Nonsensical Data

- Consider a 1-block data file

(the 1st direct pointer in the inode points to the data block)

- Adding a datablock to this file involves three changes:
 - 1) Data must be written to data block
 - 2) Direct pointer from inode should point to the data block.
 - 3) Update the data block usage bitmap.

What if something goes wrong?

Data (1)	inode (2)	bitmap (3)	Effect
no	no	yes	Inconsistent
no	yes	no	
yes	no	no	Data loss
no	yes	yes	Garbage / nonsensical data
yes	yes	no	Inconsistent
yes	no	yes	

Filesystems

Journaling

- **Goal:** Keep the file system consistent after a crash or power outage.
- **What:** Specifying on disk what changes we want to make before we make them
- **Where:** log or journal → *write-ahead logging*.
- **How:**
 - 1) Write the 3 changes as a single *transaction* → [**TBB** | iNode | Bitmap | Data | **TBE**]
 - 2) Once added to the log, we execute the transaction
 - 3) Remove transaction from the log
 - 4) If crash → execute the transactions in the log.

Filesystems

Journaling

- **Goal:** Keep the file system consistent after a crash or power outage.
- **What:** Specifying on disk what changes we want to make before we make them
- **Where:** log or journal → *write-ahead logging*.
- **How:**
 - 1) Write the 3 changes as a single *transaction* → [**TBB** | iNode | Bitmap | Data | **TBE**]
 - 2) Once added to the log, we execute the transaction
 - 3) Remove transaction from the log
 - 4) If crash → execute the transactions in the log.

Q: Any potential issue?
– disk-writting costs?
– Fail during transaction logging?

Key points

Filesystems

Pay attention to...

- Open-file tables, memory-mapped I/O
- Organization directory structure (tree, graph)
- Access methods, allocation methods
 - Continuous, linked, indexed | FAT, UNIX inode
- Free-space management capabilities
 - Fast file system → key factors behind its design.
- Journaling

Further Reading

Essential: Filesystems

- OS Course Notes - chapter 8.
 - Fast File System for UNIX* (check use case)
- Blackboard – Recorded lectures from Prof. Van Houdt.
 - Filesystemen, Sectie 3
 - Filesystemen, Sectie 4 en 5

Further Reading

Essential: Filesystems

- OS Course Notes - chapter 8.
 - Fast File System for UNIX* (check use case)
- Blackboard – Recorded lectures from Prof. Van Houdt.
 - Filesystemen, Sectie 3
 - Filesystemen, Sectie 4 en 5

Next (and final) Exercise Session

- Concurrency – 16/12/2025



Lots of energy and lots of success in your exams

Questions?



Operating Systems

[1500WETOPS]

José Oramas